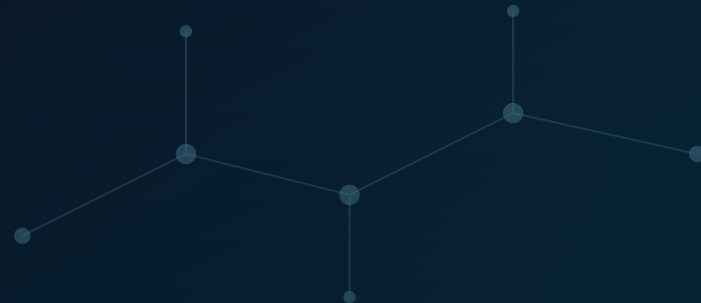


INF 345 — Fundamentals of DevOps

Lecture 6: Container Security & Module Wrap-up

Adil Akhmetov · Lesson 6



Recap — Lesson 5 (Networking & Volumes)

- Why can two containers on a user-defined network reach each other by name? (the network has built-in DNS that resolves service/container names)
- What's the difference between `docker compose down` and `down -v`? (`-v` also deletes named volumes — the data is gone)
- Why should the database publish no host port? (only the app needs to reach it, over the internal network — publishing it widens the attack surface for nothing)

Today's agenda

- ☐ Threat model: what an attacker gets when a container is compromised
- ☐ Root in a container — and how to stop being root
- ☐ Capabilities, read-only filesystems, `no-new-privileges`
- ☐ Secrets: why they never belong in an image
- ☐ Supply chain: pinning, minimal bases, scanning
- ☐ Module wrap-up: containers, weeks 3–6 — and Lab 01
- ☐ Straight into today's practice: harden a Containerfile

The scenario

Your app has a bug — say, a file-upload endpoint that lets an attacker run a command inside the container. It happens to every team eventually. The question isn't **whether** a container gets compromised.

The question is: once they're in, what can they reach? Today is about making the answer "almost nothing".

Block 1

Containers are not VMs

What actually isolates a container

- A container is a **normal Linux process** with a restricted view: namespaces (what it can *see*) and cgroups (how much it can *use*).
- There is **one kernel**, shared with the host and every other container. A VM has its own kernel; a container doesn't.
- So security is layered on top: users, capabilities, seccomp, SELinux/AppArmor, a read-only filesystem.
- Every one of those layers is something **you** can switch on or off in a Containerfile or a `run` flag.

Defense in depth: assume one layer fails. A bug in your app should not be a bug in the host.

Root in the container = root on the host?

```
docker run --rm alpine id  
# uid=0(root) gid=0(root) - the default
```

- With rootful Docker and no user namespaces, uid 0 inside **is** uid 0 on the host kernel — just fenced in by namespaces.
- One kernel bug, one mis-mounted path (`-v /:/host`), or one `--privileged` , and the fence is gone.
- **Rootless Podman** maps container root to *your* unprivileged user on the host — a big reason this course uses Podman.
- Even then: don't run as root inside. Defense in depth.

Stop being root: USER

```
FROM alpine:3.20
RUN adduser -D -u 10001 app      # create the user (alpine syntax)
COPY --from=build /out/server /server
USER 10001                      # every later RUN, and the container, run as 10001
CMD ["/server"]
```

- Prefer a **numeric uid** — Kubernetes' `runAsNonRoot` can only verify numbers, and a name that doesn't exist in `/etc/passwd` fails at start.
- Distroless images ship a ready-made one: `gcr.io/distroless/static-debian12:nonroot` (uid 65532).
- Files the app must write need the right owner: `COPY --chown=10001:10001 ...`
- Ports below 1024 traditionally need root → listen on 8080, publish `-p 80:8080` .

Check who you really are

```
docker image inspect -f '{{.Config.User}}' myimg    # what the image says
docker run --rm myimg id                            # what actually runs
docker run --rm --user 0 myimg id                   # anyone can override – at runtime
```

`USER` sets the **default**. Orchestrators can enforce it: Kubernetes `securityContext.runAsNonRoot: true` refuses to start a pod whose image would run as uid 0.

Quick check

Your Containerfile ends with `USER app`. The build succeeds, but the container exits immediately with *"unable to find user app"*. What happened?

The user was never created in the final image — a user *name* must exist in that image's `/etc/passwd`. Create it in the final stage, or use a numeric uid.

Block 2

Least privilege at runtime

Capabilities: root, split into pieces

- Linux splits root's power into ~40 **capabilities**: `NET_BIND_SERVICE` (low ports), `CHOWN`, `SYS_ADMIN` (almost everything), `NET_RAW` (raw packets, e.g. ping spoofing)...
- Docker gives containers a default set of ~14. Your web app needs **zero** of them.

```
docker run --cap-drop ALL myimg           # drop everything
docker run --cap-drop ALL --cap-add NET_BIND_SERVICE myimg # add back only
                                                    # what you need
```

`--privileged` is the opposite: **all** capabilities, all devices, no seccomp. Treat it as "this container is the host".

Read-only root filesystem

```
docker run --read-only --tmpfs /tmp myimg
```

- The image's filesystem becomes immutable at runtime — an attacker can't drop a binary, edit a config, or plant a cron job.
- Apps still need *somewhere* to write → give them exactly that: `--tmpfs /tmp` (in memory), or a volume for real data.
- If your app crashes under `--read-only`, it's telling you where it writes. That's useful information.

no-new-privileges and the hardened run

```
docker run -d -p 8080:8080 \  
  --read-only --tmpfs /tmp \  
  --cap-drop ALL \  
  --security-opt no-new-privileges \  
  myimg
```

- `no-new-privileges` : no process in the container can gain privileges later — `setuid` binaries like `su` or `sudo` stop working.
- This one command is today's practice's grading run — note there's no `--user` : the image's own `USER` has to make it non-root. If your image survives it, it is in better shape than most images on Docker Hub.
- In Compose: `read_only: true` , `cap_drop: [ALL]` , `security_opt: [no-new-privileges:true]` , `tmpfs: [/tmp]` .

Block 3

Secrets

Images are not secret

- Anyone who can **pull** an image can read **everything** in it: every layer, every `ENV`, every build step.
- Images get pushed to registries, cached on CI runners, copied to laptops. Assume an image is public.
- So a secret in an image is a leaked secret — the only question is when.

```
docker image inspect -f '{{.Config.Env}}' myimg      # ENV API_TOKEN=... in plain sight
docker history --no-trunc myimg                     # every ENV/ARG/RUN line, values included
docker save myimg -o img.tar && tar -xf img.tar      # every file, in every layer
```

Three ways secrets leak into images

Leak	Why it's in there	Fix
<code>ENV API_TOKEN=...</code>	ENV is image metadata	Pass at runtime: <code>-e</code> , <code>--env-file</code>
<code>ARG TOKEN=...</code> used in <code>RUN</code>	Build args are recorded in history	BuildKit secret mounts
<code>COPY . .</code> copies <code>.env</code>	It's just a file in a layer	<code>.dockerignore</code> , copy only what you need

`RUN rm .env` in a later step does **not** help: the file is still in the earlier layer. Layers only ever add — deleting just hides it from the final view.

Doing it right

At runtime — the secret never touches the image:

```
docker run -e API_TOKEN="$API_TOKEN" myimg  
docker run --env-file .env myimg
```

Kubernetes: a `Secret`, mounted as env or file.

At build time — when a build *needs* one (private registry, private git):

```
# syntax=docker/dockerfile:1  
RUN --mount=type=secret,id=npn \  
    NPM_TOKEN=$(cat /run/secrets/npm) npm ci
```

```
docker build --secret id=npn,src=.npmrc-token .
```

Mounted only for that one `RUN`; never written to a layer.

And `.dockerignore` with `.env`, `.git`, keys — it also makes builds faster.

Quick check

A multi-stage build does `COPY . .` — `.env` included — in the **builder** stage only, and the final stage copies just the binary. Is the secret in the shipped image?

No — only the final stage ships. But the builder's layers stay in your build cache and CI runner, so add `.dockerignore` anyway.

Block 4

Supply chain: what you build on

Pin your base images

```
FROM golang:latest           # whatever "latest" means today
FROM golang:1.23-alpine      # a version – readable, still movable
FROM golang:1.23-alpine@sha256:4a1c... # exactly these bytes, forever
```

- `latest` is just a tag someone moves. Same Containerfile, different image next week — non-reproducible builds, surprise breakage.
- A version tag is the usual balance; a **digest** is fully reproducible (bots like Renovate/Dependabot bump digests for you).
- Pin **every** stage — the builder's toolchain is part of your supply chain too.

Smaller image, smaller attack surface

Base	Size	Shell & package manager
<code>golang:1.23</code>	~800 MB	yes — and a compiler
<code>alpine:3.20</code>	~8 MB	<code>sh</code> , <code>apk</code>
<code>gcr.io/distroless/static-debian12</code>	~2 MB	no shell at all
<code>scratch</code>	0 MB	nothing — not even <code>/tmp</code>

- Every package you ship is a package that can have a CVE.
- No shell means an attacker who gets code execution can't just `sh` their way around.
- Multi-stage builds (week 4) are how you get here: build fat, ship thin.

Scanning: know what you ship

```
trivy image --severity HIGH,CRITICAL myimg
```

- Scanners (Trivy, Grype, Docker Scout) match the packages in your image against public vulnerability databases (CVEs).
- Run them in CI — a new CVE can appear for an image you built months ago.
- Most findings in a fat image are in packages your app never uses. Minimal bases make the report short and meaningful.
- Related ideas you'll meet at work: **SBOMs** (a bill of materials for an image) and **image signing** (cosign) — proof of what and who.

The hardening checklist

- ☐ Every `FROM` pinned — tag or digest, never `latest`
- ☐ Multi-stage: the final image has the app and nothing else
- ☐ Non-root `USER` , numeric uid
- ☐ No secrets in `ENV` , `ARG` , or copied files — `.dockerignore`
- ☐ Runs with `--read-only --tmpfs /tmp --cap-drop ALL --security-opt no-new-privileges`
- ☐ Scanned in CI

This is today's practice, line by line.

Block 5

Module wrap-up: containers

Weeks 3–6 in one slide

Week	You learned to...
3	run, inspect and stop containers; images vs containers
4	write Containerfiles; layers, caching, multi-stage
5	connect containers; volumes; Compose
6	ship them safely

The whole module, as one sentence:

"Build a small, pinned, non-root image without secrets; run it with only the network access and storage it needs; describe all of that in a file that lives in git."

Lab 01 — Red Hat Academy DO188

- **Due today (week 6), end of day.** Graded from your DO188 completion on Red Hat Academy: completion × 15 points.
- Maru syncs your RHA progress every hour — check **My grades** on your dashboard to see exactly what we see.
- Not finished? An unfinished lab counts at the percentage you've reached — every guided exercise you complete tonight still counts.
- Next module (week 7+): **automation with Ansible** — Lab 02 is RHA RH294, due week 10.

Block 6

Today's practice

Today's practice — harden a Containerfile

Practice 06 gives you a Go app and a Containerfile that **works** — but:

1. FROM `golang:latest` — unpinned, ~1 GB, ships a compiler
2. No `USER` — runs as root
3. ENV `API_TOKEN=...` — a secret baked in
4. COPY `. .` — copies `.env` too

Fix all of it. Don't touch `main.go`.

```
docker build -t p06 -f Containerfile .
docker run --rm -p 8080:8080 \
  --read-only --tmpfs /tmp \
  --cap-drop ALL \
  --security-opt no-new-privileges \
  -e API_TOKEN=x p06
curl localhost:8080
# INF345 Practice 06 OK uid=65532 token=set
```

Graded automatically: pinned bases, size under 50 MB, non-root, token nowhere in the image, works hardened.

By the end of this lesson, you should be able to

- ☐ Explain why a root process in a container is a risk to the host
- ☐ Write a Containerfile that runs as a non-root, numeric uid
- ☐ Run a container with no capabilities on a read-only filesystem
- ☐ Find a secret in an image — and keep one out of it
- ☐ Choose and pin a minimal base image, and scan the result

Before next lecture

- ☐ Finish today's practice if you didn't wrap it up in session — same Maru flow as last week
- ☐ **Lab 01 (RHA DO188) is due today** — check "My grades" in Maru
- ☐ Start RHA **RH294** — Lab 02 is due in week 10

That closes the containers module. Next: automating the machines the containers run on.

Next lecture

Configuration management & Ansible basics — inventories, ad-hoc commands, and your first playbook.